

Device Monitoring Application

Final Project Scope & Technical Plan — Android, Multi-Tenant

1. Purpose & Use Cases

- **Parental Control:** Parents monitoring their minor children's devices.
- **Employee Transparency:** Company monitoring employee activity for security/productivity purposes, with explicit written consent (device not company-owned, but usage is designated company-purpose only).
- **Product Type:** Internal multi-tenant product (not for public sale); architecture supports multiple isolated tenant accounts (families / companies).

2. Platform

Android only for Phase 1. iOS support is on hold due to platform sandboxing restrictions that prevent equivalent feature parity without jailbreak or credential-based (iCloud) access, which is out of scope.

3. Core Non-Negotiable Principles

- App icon is always visible on the target device — no hidden/stealth operation.
- Mandatory install-time consent screen before any permission is granted.
- No call recording, in any form (live, post-call, root-based, or network-level).
- No reading of message content from WhatsApp, Telegram, Instagram, or any other messaging app.
- No live or hidden audio/video streaming.
- No silent/remote photo capture.

These exclusions exist because such features would capture the private communications of third parties (callers, chat contacts) who never consented, and are the defining technical signature of stalkerware — regardless of the device owner's own consent.

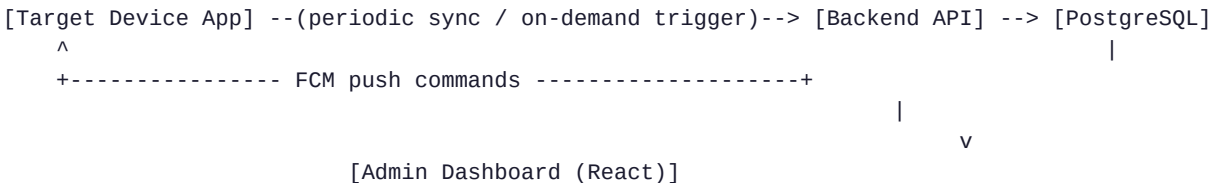
4. Feature Scope & Technical Working

#	Feature	How It Works
1	On-Demand Location Check	Admin clicks 'Check Location Now' on dashboard -> Backend sends FCM push -> App wakes briefly, fetches GPS via FusedLocationProviderClient -> Uploads to backend -> Dashboard shows pin on map. No continuous tracking, no persistent notification.
2	Geofencing	Admin defines a boundary (lat/lng + radius) -> App registers it with Android's native GeofencingClient API -> OS monitors boundary at hardware level (near-zero battery cost) -> On crossing, OS wakes app briefly -> Event uploaded to backend -> Dashboard alert triggered.
3	App Usage / Screen Time	Periodic background job (WorkManager, every 2-4 hrs) reads UsageStatsManager -> Collects per-app time spent -> Buffered in local Room DB -> Synced to backend when online -> Dashboard shows daily/weekly charts per app.
4	Website Browsing History	Periodic job reads browser history via supported browsers' content providers -> Synced same as above -> Dashboard shows a visited-sites timeline.
5	App / Website Blocking	Admin sets a blocklist/allowlist and schedule on dashboard -> Synced to device via FCM -> App enforces restrictions using Accessibility Service (disclosed at consent) or Device Admin API.
6	SMS Log (metadata)	Periodic job reads SMS content provider (sender/receiver number, timestamp) -> Synced to backend -> Dashboard shows the log list.
7	Call Log (metadata only)	Periodic job reads CallLog content provider (number, duration, timestamp — no audio) -> Synced to backend -> Dashboard shows call history.
8	Remote Lock	Admin clicks 'Lock Device' -> Backend sends FCM push command -> App (registered as Device Admin) calls lockNow() -> Device screen locks immediately.
9	File Manager (view only)	Admin clicks 'View Files' -> Backend sends FCM push -> App lists accessible files via Storage Access Framework -> List sent to backend -> Dashboard shows folder tree.
10	File Download (on-demand, no server storage)	Admin clicks 'Download' on a file -> Backend sends FCM push -> App opens a live WebSocket to backend -> File streamed in chunks directly to admin's browser -> Nothing persisted on the server; connection closes after transfer.
11	Installed Apps List	Periodic job reads PackageManager -> List of installed apps synced to backend -> Dashboard shows inventory.

5. System Components & Tech Stack

Component	Technology	Role
Android App	Kotlin, Room (local buffer), WorkManager (periodic sync), FCM (push commands), GeofencingClient (native geofence)	Data collection + on-demand actions
Backend API	Node.js + Express	REST API, auth, command dispatch, data storage
Database	PostgreSQL (tenant_id row-level isolation)	Multi-tenant data store for all logs/config
Admin Dashboard	React + TypeScript, Tailwind CSS, Recharts	Tenant/device management, reports, live actions
Push/Command Channel	Firebase Cloud Messaging (FCM)	Triggers on-demand actions: location check, file download, lock, config sync
File Transfer	WebSocket (temporary, no persistence)	Direct device-to-browser file relay

6. Data Flow Overview



7. Battery & Notification Strategy

- No continuous foreground service is run → no persistent notification required.
- Location: fetched on-demand only, triggered from the dashboard.
- Geofencing: handled by Android's native OS-level GeofencingClient (near-zero battery cost).
- App usage / SMS / call logs: periodic short background jobs (few seconds each, every 2-4 hours).

8. On-Device Consent Flow

1. Target user installs the APK (visible app icon, standard app listing).
2. On first launch, a mandatory consent screen explains what data is collected and why.
3. User must tap 'Allow' before any permission (Location, Usage Access, SMS, Storage, etc.) is requested.
4. Only after consent is granted does the device register with the backend and begin scheduled syncs.

9. Explicitly Excluded Features (Final)

- Call recording — in any form (live, post-call, root-based, or network/carrier-level interception).
- Message content interception — WhatsApp, Telegram, Instagram, or any other messaging app.
- Live or hidden audio/video streaming.
- Silent/remote photo capture.
- Hidden app icon or fully invisible background operation.

Reason for exclusion: each of the above would capture the private communications or imagery of a third party (the other party on a call, a chat contact, or the person being photographed) who never consented, and/or would make the monitoring covert to the device owner. These patterns match the technical definition of stalkerware used by security vendors (e.g. Google Play Protect) and, in most jurisdictions, wiretapping / interception law, regardless of the device owner's own consent or the purchasing organization's internal policy.

10. Next Steps

1. Database schema design (tenants, devices, users, feature_flags, location_logs, app_usage_logs, sms_logs, call_logs, alerts).
2. API endpoint specification.
3. Android app module structure.
4. Admin dashboard page structure & wireframes.